Documento de Arquitetura – Sistema de Biblioteca

Versão 1.0 · Documento de decisões, arquitetura e estrutura do projeto

1. Visão geral

Sistema desktop para gerenciamento de uma biblioteca de porte médio (universitária ou municipal), executado em **um único computador Linux**, operado apenas pelo **administrador** (com login). O sistema controla acervo, leitores, empréstimos, devoluções e renovações, além de gerar estatísticas exportáveis. Os dados ficam em um banco **SQLite local**, com backup em **HDD externo**.

2. Decisões tomadas

Tema	Decisão
Porte	Biblioteca média (universitária ou municipal)
Usuários	Somente administrador (login único)
Plataforma	Desktop, Linux
Computadores simultâneos	1
Front-end	React
Back-end	Python com FastAPI
Banco de dados	SQLite (arquivo local)
Empacotamento	PyWebView + PyInstaller, instalado como aplicativo Linux (.deb ou script de instalação)
Funcionalidades	Cadastro de acervo, empréstimo/devolução/renovação, relatórios e estatísticas
Fora do escopo (v1)	Multas financeiras, reservas, autoatendimento de leitores, acesso por rede
Regras de empréstimo	Diferentes por tipo de leitor (aluno, professor, público)

Tema	Decisão
Atraso	Alertar e bloquear novos empréstimos até a devolução
Importação	Acervo via CSV/XLSX
Estatísticas	Livros mais e menos emprestados, alunos com mais empréstimos (com quantidade), filtro por ano/sala; exportação em CSV e XLSX
Backup	Em HDD externo

3. Requisitos

3.1 Funcionais

- **RF01** Login do administrador.
- **RF02** Cadastrar, editar, consultar e inativar obras (livros, periódicos, mídias).
- **RF03** Cadastrar exemplares com código de barras/tombo e controlar o status (disponível, emprestado, perdido, danificado).
- **RF04** Cadastrar, editar e consultar leitores, com tipo (aluno, professor, público). Alunos têm matrícula, ano e sala.
- **RF05** Configurar a política de empréstimo por tipo de leitor (prazo, limite de itens, número de renovações).
- **RF06** Registrar empréstimo, devolução e renovação, aplicando as regras da política.
- **RF07** Detectar atrasos, exibir alertas e bloquear novos empréstimos do leitor em atraso.
- **RF08** Importar acervo por CSV/XLSX, com prévia e relatório de erros.
- **RF09** Gerar relatórios e estatísticas e exportar em CSV e XLSX.
- **RF10** Realizar backup do banco em HDD externo (manual e agendado) e restaurar a partir dele.

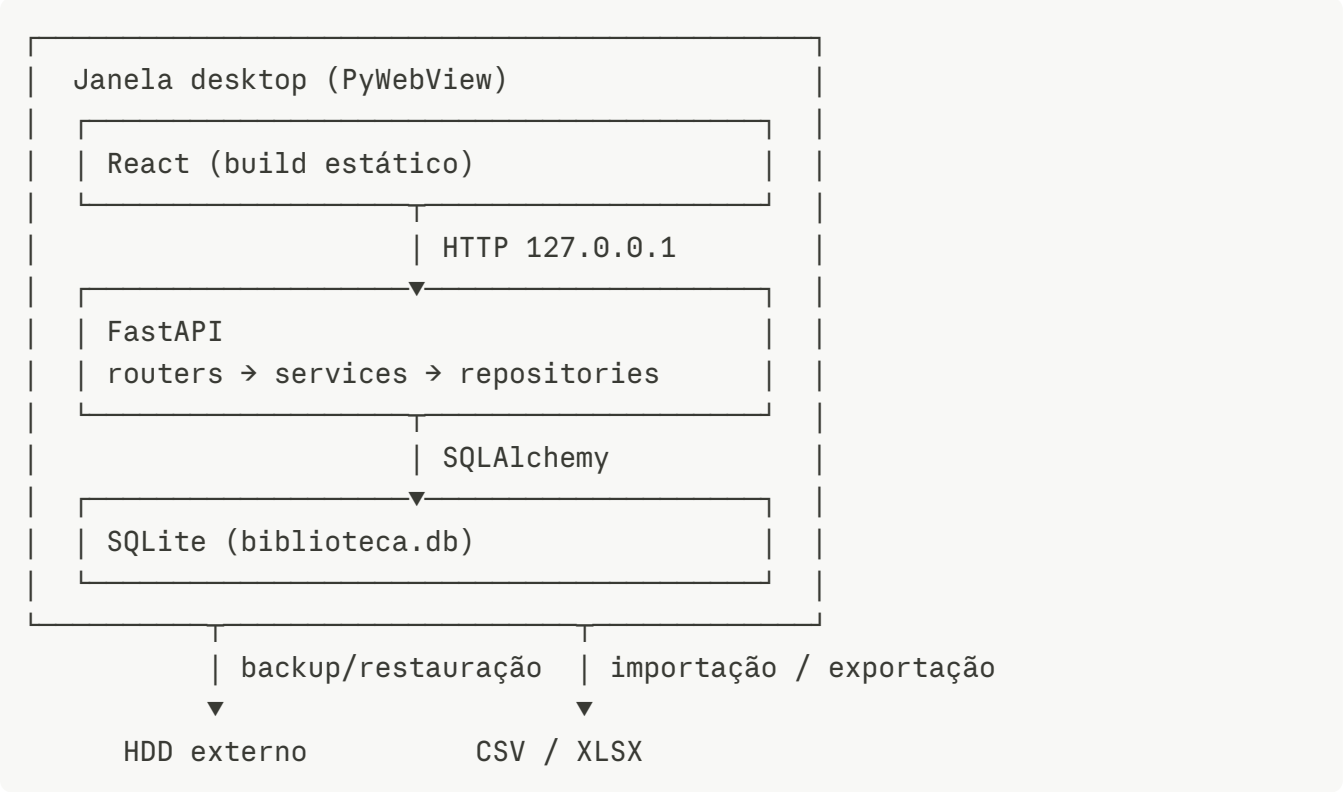
3.2 Não funcionais

- **RNF01** Funcionar offline, sem dependência de internet.
- **RNF02** A API escuta somente em `127.0.0.1`.
- **RNF03** Senhas armazenadas com hash (argon2 ou bcrypt).
- **RNF04** Os dados do usuário ficam fora da pasta do programa, para sobreviverem a atualizações.

- **RNF05** Esquema do banco versionado com migrações.
- **RNF06** Regras de negócio cobertas por testes automatizados.

4. Arquitetura

Aplicação monolítica local, em camadas, empacotada como app desktop.



4.1 Camadas do back-end

Camada	Responsabilidade
routers	Endpoints HTTP, validação de entrada/saída (schemas), sem regra de negócio
services	Regras de negócio (empréstimo, atraso, importação, relatórios, backup)
repositories	Acesso ao banco via SQLAlchemy
models	Entidades do banco
schemas	Modelos Pydantic de entrada e saída
core	Configuração, conexão com o banco, segurança e autenticação

4.2 Front-end

React com páginas por módulo, camada `services/api` para chamadas HTTP, hooks para estado e componentes reutilizáveis (tabelas, formulários, modais, alertas). Roteamento com React Router.

5. Estrutura de pastas

```
biblioteca/
├── backend/
│   ├── app/
│   │   ├── main.py                # cria o app, monta rotas e arquivos
estáticos
│   │   ├── core/
│   │   │   ├── config.py          # caminhos (DATA_DIR, DB_PATH),
constantes
│   │   │   ├── database.py        # engine, sessão, WAL e foreign keys
│   │   │   ├── security.py        # hash de senha, token
│   │   │   └── deps.py            # dependências (sessão, admin
autenticado)
│   │   ├── models/
│   │   │   ├── usuario_admin.py
│   │   │   ├── tipo_leitor.py
│   │   │   ├── leitor.py
│   │   │   ├── obra.py
│   │   │   ├── exemplar.py
│   │   │   ├── emprestimo.py
│   │   │   └── logs.py            # log_backup, log_importacao
│   │   ├── schemas/
│   │   │   ├── auth.py
│   │   │   ├── leitor.py
│   │   │   ├── obra.py
│   │   │   ├── exemplar.py
│   │   │   ├── emprestimo.py
│   │   │   └── relatorio.py
│   │   ├── repositories/
│   │   │   ├── leitor_repo.py
│   │   │   ├── obra_repo.py
│   │   │   ├── exemplar_repo.py
│   │   │   ├── emprestimo_repo.py
│   │   │   └── relatorio_repo.py
│   │   ├── services/
│   │   │   ├── auth_service.py
│   │   │   ├── acervo_service.py
│   │   │   ├── leitor_service.py
```

```
|   |   |   ├── emprestimo_service.py   # regras de empréstimo e renovação
|   |   |   ├── atraso_service.py       # detecção e bloqueio por atraso
|   |   |   ├── importacao_service.py
|   |   |   ├── relatorio_service.py
|   |   |   ├── exportacao_service.py   # CSV e XLSX
|   |   |   └── backup_service.py
|   |   ├── routers/
|   |   |   ├── auth.py
|   |   |   ├── obras.py
|   |   |   ├── exemplares.py
|   |   |   ├── leitores.py
|   |   |   ├── tipos_leitor.py
|   |   |   ├── emprestimos.py
|   |   |   ├── importacao.py
|   |   |   ├── relatorios.py
|   |   |   └── backup.py
|   |   └── static/                 # build do React copiado aqui
|   ├── migrations/                 # Alembic
|   ├── tests/
|   |   ├── test_emprestimo.py
|   |   ├── test_atraso.py
|   |   ├── test_importacao.py
|   |   └── test_relatorios.py
|   ├── alembic.ini
|   └── requirements.txt
├── frontend/
|   ├── public/
|   ├── src/
|   |   ├── pages/
|   |   |   ├── Login.jsx
|   |   |   ├── Dashboard.jsx
|   |   |   ├── Acervo.jsx
|   |   |   ├── Leitores.jsx
|   |   |   ├── Emprestimos.jsx
|   |   |   ├── Importacao.jsx
|   |   |   ├── Relatorios.jsx
|   |   |   └── Configuracoes.jsx   # políticas, backup
|   |   ├── components/
|   |   ├── services/               # api.js e um arquivo por módulo
|   |   ├── hooks/
|   |   ├── routes/
|   |   ├── App.jsx
|   |   └── main.jsx
|   ├── package.json
|   └── vite.config.js
├── desktop/
```

```
|   ├── launcher.py                # sobe o FastAPI e abre a janela
PyWebView
|   ├── biblioteca.spec            # configuração do PyInstaller
|   ├── biblioteca.desktop         # atalho do menu Linux
|   ├── icone.png
|   └── build/
|       ├── build.sh               # React → static → PyInstaller
|       ├── install.sh             # instala em /opt e cria o atalho
|       └── make_deb.sh            # gera o pacote .deb
├── docs/
|   ├── arquitetura.md
|   └── modelo-er.md
├── .gitignore
└── README.md
```

5.1 Onde ficam os dados do usuário

```
~/.local/share/biblioteca/
├── biblioteca.db        # banco SQLite
├── logs/
└── exportacoes/         # arquivos gerados (opcional)
```

O banco nunca fica dentro da pasta do programa, então atualizar ou reinstalar o app não apaga os dados.

6. Modelo de dados

Tabela	Campos principais
`usuario_admin`	id, usuario, senha_hash, criado_em
`tipo_leitor`	id, nome (aluno, professor, público), prazo_dias, limite_emprestimos, max_renovacoes
`leitor`	id, nome, matricula, tipo_leitor_id, ano, sala, contato, ativo
`obra`	id, titulo, autor, editora, ano_publicacao, isbn, categoria, tipo_material (livro, periódico, mídia), ativo
`exemplar`	id, obra_id, codigo_barras (único), status

Tabela	Campos principais
`emprestimo`	id, exemplar_id, leitor_id, data_emprestimo, data_prevista, data_devolucao, renovacoes, ano_leitor, sala_leitor
`log_backup`	id, data, destino, status, mensagem
`log_importacao`	id, data, arquivo, linhas_ok, linhas_erro, detalhes

Relações: uma obra tem vários exemplares; um leitor tem vários empréstimos; um exemplar tem vários empréstimos ao longo do tempo (apenas um em aberto); um tipo de leitor tem vários leitores.

Snapshot de ano e sala: `ano_leitor` e `sala_leitor` guardam o valor no momento do empréstimo. Assim, as estatísticas por ano/sala continuam corretas mesmo que o aluno mude de turma depois.

7. Regras de negócio

7.1 Empréstimo

O empréstimo só é permitido quando **todas** as condições são verdadeiras:

1. O leitor está ativo.
2. O exemplar está com status disponível.
3. O leitor não possui nenhum empréstimo em atraso.
4. O leitor não atingiu o limite de itens do seu tipo.

A data prevista de devolução é calculada pelo prazo do tipo de leitor.

7.2 Renovação

Permitida quando o empréstimo não está atrasado e o número de renovações é menor que o máximo do tipo de leitor. O novo prazo conta a partir da data da renovação, usando o prazo do tipo de leitor.

7.3 Devolução

Registra a data de devolução e volta o exemplar para disponível. Se havia atraso, o bloqueio do leitor deixa de existir quando não restar nenhum item atrasado.

7.4 Atraso

Um empréstimo está em atraso quando não foi devolvido e a data prevista é anterior à data atual. O sistema exibe alertas no painel, destaca os leitores em atraso e **bloqueia novos empréstimos** até a devolução. Não há multa financeira na versão 1.

7.5 Políticas iniciais (editáveis)

Tipo de leitor	Prazo	Limite de itens	Renovações
Aluno	7 dias	3	2
Professor	15 dias	5	3
Público	7 dias	2	1

Os valores acima são sugestões e podem ser alterados na tela de configurações.

8. API (principais endpoints)

Método	Rota	Descrição
POST	`/api/auth/login`	Login do administrador
GET/POST	`/api/obras`	Listar e cadastrar obras
GET/PUT/DELETE	`/api/obras/{id}`	Consultar, editar e inativar obra
POST	`/api/exemplares`	Cadastrar exemplar
PATCH	`/api/exemplares/{id}/status`	Alterar status do exemplar
GET/POST	`/api/leitores`	Listar e cadastrar leitores
GET/PUT	`/api/leitores/{id}`	Consultar e editar leitor
GET/PUT	`/api/tipos-leitor`	Consultar e editar políticas
POST	`/api/emprestimos`	Registrar empréstimo
POST	`/api/emprestimos/{id}/devolucao`	Registrar devolução
POST	`/api/emprestimos/{id}/renovacao`	Renovar empréstimo
GET	`/api/emprestimos/atrasados`	Listar atrasos

Método	Rota	Descrição
POST	`/api/importacao/previa`	Validar arquivo e mostrar prévia
POST	`/api/importacao/confirmar`	Executar a importação
GET	`/api/relatorios/mais-emprestados`	Livros mais emprestados
GET	`/api/relatorios/menos-emprestados`	Livros menos emprestados
GET	`/api/relatorios/alunos-top`	Alunos com mais empréstimos (filtros ano/sala)
GET	`/api/relatorios/{nome}/exportar?formato=csv|xlsx`	Exportar relatório
POST	`/api/backup`	Fazer backup no HDD
POST	`/api/backup/restaurar`	Restaurar backup
GET	`/api/backup/historico`	Histórico de backups

9. Telas do front-end

- **Login:** usuário e senha do administrador.
- **Dashboard:** empréstimos do dia, itens em atraso, atalhos para empréstimo e devolução.
- **Acervo:** busca, cadastro de obras e exemplares, status.
- **Leitores:** busca, cadastro, histórico de empréstimos e situação (em atraso ou bloqueado).
- **Empréstimos:** balcão rápido com leitura de código de barras, devolução e renovação.
- **Importação:** envio do arquivo, prévia, confirmação e relatório de erros.
- **Relatórios:** filtros por período, ano e sala; botões de exportar CSV e XLSX.
- **Configurações:** políticas por tipo de leitor, backup e restauração, alteração de senha.

10. Relatórios e estatísticas

- Livros mais emprestados e menos emprestados, por período.
- Alunos com mais empréstimos, com a quantidade, filtráveis por ano e sala.
- Empréstimos por período.

- Atrasos atuais e histórico de atrasos.
- Todos exportáveis em **CSV** (biblioteca padrão do Python) e **XLSX** (openpyxl).

11. Importação do acervo

1. O usuário escolhe o arquivo CSV ou XLSX.
2. O sistema valida colunas obrigatórias e formatos e exibe uma prévia com linhas válidas e com erro.
3. Ao confirmar, a importação roda em **uma transação**: se algo falhar, nada é gravado pela metade.
4. É gerado um relatório com linhas importadas e erros, registrado em `log_importacao`.

Colunas sugeridas: título, autor, editora, ano, ISBN, categoria, tipo de material, código de barras/tombo, quantidade de exemplares.

12. Backup

- Usa a **API de backup do SQLite**, que é segura mesmo com o banco aberto (não se copia o arquivo diretamente).
- Destino: pasta no HDD externo, escolhida na tela de configurações. Os arquivos levam data e hora no nome.
- Após copiar, o sistema valida a integridade do arquivo gerado.
- Backup manual com um clique e, opcionalmente, agendado (por exemplo, ao fechar o app ou diariamente).
- Se o HDD não estiver conectado, o sistema avisa e registra a falha em `log_backup`.
- A restauração substitui o banco atual, mantendo uma cópia de segurança do estado anterior.

13. Segurança

- API ligada apenas em `127.0.0.1`, em porta livre escolhida na inicialização.
- Senha do administrador com hash (argon2 ou bcrypt); definição da senha no primeiro uso.
- Token de sessão com expiração; rotas da API protegidas, exceto login.
- Validação de todas as entradas com Pydantic.
- Arquivos importados são validados por tipo e tamanho.

14. Empacotamento e execução como aplicativo Linux

14.1 Dependências do sistema (Debian/Ubuntu)

```
sudo apt install python3-gi gir1.2-webkit2-4.1 python3-venv
```

14.2 Fluxo de build

1. `npm run build` no `frontend/`.
2. Copiar `frontend/dist` para `backend/app/static`.
3. O FastAPI serve o build com `StaticFiles` montado depois das rotas da API.
4. O PyInstaller (modo `--onedir`) gera o executável a partir de `desktop/launcher.py`.
5. O script de instalação copia para `/opt/biblioteca/` e registra o atalho no menu.

14.3 Launcher

```
import threading, socket, time
import uvicorn, webview
from app.main import app

def porta_livre():
    with socket.socket() as s:
        s.bind(("127.0.0.1", 0))
        return s.getsockname()[1]

PORTA = porta_livre()

def servidor():
    uvicorn.run(app, host="127.0.0.1", port=PORTA, log_level="warning")

threading.Thread(target=servidor, daemon=True).start()
time.sleep(1)  # melhoria: aguardar a porta responder

webview.create_window("Biblioteca", f"http://127.0.0.1:{PORTA}",
                      width=1280, height=800)
webview.start()
```

14.4 Atalho no menu (`biblioteca.desktop`)

```
[Desktop Entry]
Type=Application
Name=Biblioteca
Comment=Sistema de gerenciamento de biblioteca
Exec=/opt/biblioteca/biblioteca
Icon=/opt/biblioteca/icone.png
Terminal=false
Categories=Office;Education;
```

14.5 Distribuição

Recomendação: **pacote .deb** (declara as dependências e instala com duplo clique) ou o **script `install.sh`**. AppImage fica como alternativa, mas empacotar GTK/WebKit dentro dele é trabalhoso e só compensa se for rodar em várias distribuições.

Se o WebKit der problema, a alternativa é abrir o sistema com `chromium --app=http://127.0.0.1:PORTA`, mantendo o mesmo back-end.

15. Tecnologias e bibliotecas

Camada	Tecnologias
Back-end	Python 3.11+, FastAPI, Uvicorn, SQLAlchemy 2, Alembic, Pydantic, passlib (argon2/bcrypt), openpyxl, pytest
Front-end	React, Vite, React Router, biblioteca de componentes à escolha (ex.: MUI ou Ant Design), gráficos com Recharts
Desktop	PyWebView, PyInstaller, dpkg-deb ou fpm
Banco	SQLite com modo WAL e chaves estrangeiras ativas

16. Estratégia de testes

- **Unitários (pytest):** regras de empréstimo, renovação, devolução e atraso; são o núcleo do sistema.
- **Integração:** endpoints principais com banco SQLite em memória.
- **Importação:** arquivos válidos, inválidos e parcialmente inválidos.

- **Relatórios:** conferência dos números com dados de exemplo.
- **Backup:** gerar, validar e restaurar um backup.
- **Manual:** instalação limpa em Linux, abertura pelo menu e fluxo completo de balcão.

17. Roadmap

Etapa	Entrega
1	Base do projeto, banco com migrações, login do administrador
2	CRUD de acervo (obras e exemplares) e leitores, políticas por tipo
3	Empréstimo, devolução, renovação e regras de atraso
4	Importação CSV/XLSX
5	Relatórios, estatísticas e exportação
6	Backup e restauração no HDD
7	Empacotamento Linux (.deb), testes finais e documentação

18. Evoluções futuras (fora da v1)

- Multas financeiras e reservas.
- Perfis adicionais (atendente) e auditoria de ações.
- Acesso de leitores por consulta de catálogo.
- Operação em rede com vários computadores (migrando para PostgreSQL).
- Impressão de carteirinhas e etiquetas de código de barras.